Worksheet No. - 6

Student Name: Sujal Dhiman **UID: 25MCA20186**
Branch: MCA **Section/Group: 1A**
Semester: 2nd **Date of Performance: 17/02/2026**
Subject Name: TECHNICAL TRAINING **Subject Code:**25CAP-652

Aim/Overview of the practical:

Learn how to create, query, and manage views in SQL to simplify database queries and provide a layer of abstraction for end-users.

Objective:

- **Data Abstraction:** To understand how to hide complex table joins and calculations behind a simple virtual table interface.
- **Enhanced Security:** To learn how to restrict user access to sensitive columns by providing views instead of direct table access.
- **Query Simplification:** To master the creation of views that pre-join multiple tables, making reporting easier for non-technical users.
- **View Management:** To understand the syntax for creating, altering, and dropping views, as well as the naming conventions required for efficient data access.

Theory:

A View is essentially a virtual table based on the result-set of an SQL statement. It does not contain data of its own but dynamically pulls data from the underlying "base tables".

1. **Simple Views:** Created from a single table without any aggregate functions or grouping. These are often updatable.
2. **Complex Views:** Created from multiple tables using JOINs, or including GROUP BY and aggregate functions. These provide a consolidated summary of the database.
3. **Security Layer:** In enterprise environments, views are used to grant permissions on specific subsets of data. For example, a "SalaryView" might exclude the "Employee_SSN" or "Home_Address" columns for privacy.
4. **Benefits:** They simplify the user experience, ensure data consistency across reports, and reduce the risk of accidental data modification by providing read-only abstractions.

Input/Apparatus Used:

- Oracle
- MS SSMS
- PostgreSQL

Procedure/Algorithm/Code :

```
CREATE TABLE employees (
   emp_id SERIAL PRIMARY KEY,
   emp_name VARCHAR(50),
   department VARCHAR(50),
   salary INT
);

INSERT INTO employees (emp_name, department, salary) VALUES
('Amit', 'IT', 60000),
('Riya', 'HR', 45000),
('Sohan', 'IT', 70000),
('Neha', 'Finance', 40000);


-- create view for IT employees
CREATE VIEW it_employees AS
SELECT emp_id, emp_name, salary
FROM employees
WHERE department = 'IT';

-- query the view
SELECT * FROM it_employees;

-- update data using view
UPDATE it_employees
SET salary = salary + 5000
WHERE emp_id = 1;

-- create view for high salary employees
CREATE VIEW high_salary AS
SELECT emp_name, department, salary
FROM employees
WHERE salary > 50000;

-- query second view
SELECT * FROM high_salary;

-- create read-only view
CREATE VIEW readonly_view AS
```

```
SELECT emp_name, salary
FROM employees
WITH CHECK OPTION;

-- drop view
DROP VIEW it_employees;
```

Output:

	emp_id integer	emp_name character varying (50)	salary integer
1	1	Amit	60000
2	3	Sohan	70000

It_employees view

	emp_name character varying (50)	department character varying (50)	salary integer
1	Sohan	IT	70000
2	Amit	IT	65000

High_salary view

Learning outcomes (What I have learnt):

- **Abstraction Proficiency:** Students will be able to create and query views to simplify efficient data access and abstraction.
- **Security Implementation:** Students will understand how to use views for data masking and providing restricted access to sensitive information.
- **Syntactic Accuracy:** Students will demonstrate the correct syntax for creating and querying views, ensuring logical clarity in naming conventions.
- **Real-world Application:** Students will be able to design views for practical domains like Library Management Systems or Payroll Systems to demonstrate functionality.